```
pip install minisom
```

```
Requirement already satisfied: minisom in /usr/local/lib/python3.12/dist-packages (2.3.5)
```

```
from minisom import MiniSom
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

```
# Please upload the 'Mall Customers.xlsx' file to your Colab environment (e.g., to the /content/ directory) before running t
data = pd.read_excel("/content/Mall Customers.xlsx")
print(data.head())
```

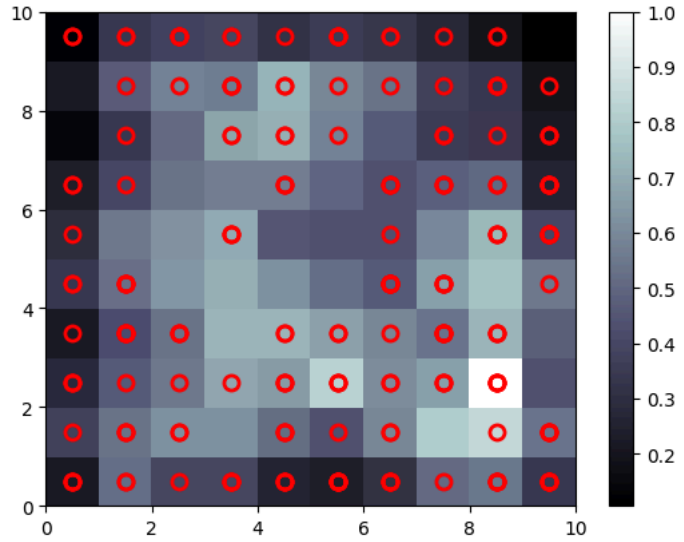
```
   CustomerID Gender  Age   Education  Marital Status  Annual Income (k$)  \
0           1      M   19  High School        Married                  15
1           2      M   21     Graduate         Single                  15
2           3      F   20     Graduate        Married                  16
3           4      F   23  High School        Unknown                  16
4           5      F   31   Uneducated        Married                  17

   Spending Score (1-100)
0                      39
1                      81
2                       6
3                      77
4                      40
```

```
X_som = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].values

scaler = StandardScaler()
X_som = scaler.fit_transform(X_som)
```

```
som = MiniSom(x=10, y=10, input_len=3, sigma=1.0, learning_rate=0.5)
som.random_weights_init(X_som)
```

```
som.train_random(data=X_som, num_iteration=1000)
```

```
from pylab import bone, pcolor, colorbar, plot, show

bone()
pcolor(som.distance_map().T)
colorbar()

for i, x in enumerate(X_som):
    w = som.winner(x)
    plot(w[0]+0.5, w[1]+0.5, 'o', markerfacecolor='None',
         markeredgecolor='r', markersize=8, markeredgewidth=2)

show()
```




```
X = data[['Annual Income (k$)', 'Spending Score (1-100)']]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

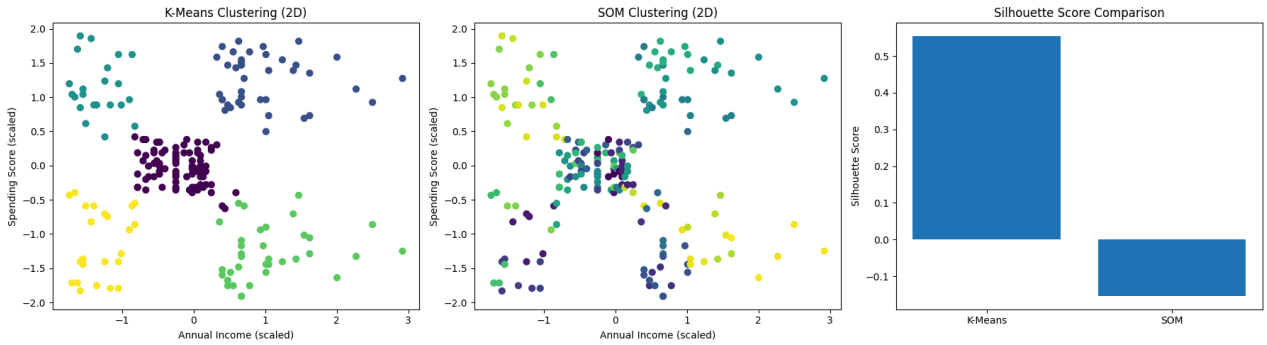
```
kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

data['Cluster'] = clusters
```

```
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=clusters, cmap='viridis')
plt.xlabel("Annual Income (scaled)")
plt.ylabel("Spending Score (scaled)")
plt.title("Customer Segmentation using K-Means")
plt.show()
```


```
# ================================
# Imports
# ================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
from minisom import MiniSom


# ================================
# Load Dataset
# ================================
data = pd.read_excel("/content/Mall Customers.xlsx")


# ================================
# Feature Selection
# ================================
X_2d = data[['Annual Income (k$)', 'Spending Score (1-100)']].values
X_som = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].values


# ================================
# Scaling
# ================================
scaler = StandardScaler()
X_2d_scaled = scaler.fit_transform(X_2d)
X_som_scaled = scaler.fit_transform(X_som)


# ================================
# K-Means
# ================================
kmeans = KMeans(n_clusters=5, random_state=42)
kmeans_labels = kmeans.fit_predict(X_2d_scaled)


# ================================
# Train SOM
# ================================
som = MiniSom(
    x=10, y=10,
    input_len=3,
    sigma=1.0,
    learning_rate=0.5,
    random_seed=42
)

som.random_weights_init(X_som_scaled)
som.train_random(X_som_scaled, num_iteration=1000)


# ================================
# Convert SOM mapping to cluster labels
# ================================
som_labels = np.array([
    som.winner(x)[0] * 10 + som.winner(x)[1]
    for x in X_som_scaled
])


# ================================
# Evaluation Metrics
# ================================
silhouette_kmeans = silhouette_score(X_2d_scaled, kmeans_labels)
silhouette_som = silhouette_score(X_2d_scaled, som_labels)


# ================================
# Plotting (3 graphs)
# ================`
fig, axes = plt.subplots(1, 3, figsize=(18, 5))


# ---- Plot 1: K-Means (2D Scatter) ----
axes[0].scatter(
    X_2d_scaled[:, 0],
    X_2d_scaled[:, 1],
    c=kmeans_labels,
    cmap='viridis',
    s=40
)
axes[0].set_title("K-Means Clustering (2D)")
axes[0].set_xlabel("Annual Income (scaled)")
axes[0].set_ylabel("Spending Score (scaled)")


# ---- Plot 2: SOM (2D Scatter) ----
```

```
axes[1].scatter(
    X_2d_scaled[:, 0],
    X_2d_scaled[:, 1],
    c=som_labels,
    cmap='viridis',
    s=40
)
axes[1].set_title("SOM Clustering (2D)")
axes[1].set_xlabel("Annual Income (scaled)")
axes[1].set_ylabel("Spending Score (scaled)")


# ---- Plot 3: Metric Comparison ----
axes[2].bar(
    ['K-Means', 'SOM'],
    [silhouette_kmeans, silhouette_som]
)
axes[2].set_title("Silhouette Score Comparison")
axes[2].set_ylabel("Silhouette Score")


plt.tight_layout()
plt.show()


# ================================
# Print Scores (for report)
# ================================
print("Silhouette Score:")
print(f"K-Means : {silhouette_kmeans:.3f}")
print(f"SOM     : {silhouette_som:.3f}")
```



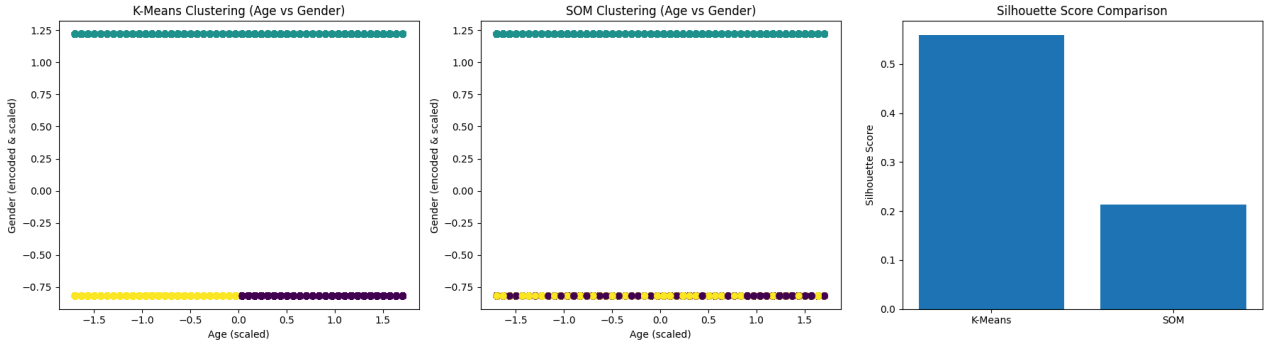

```
Silhouette Score:
K-Means : 0.555
SOM     : -0.155
```

```
# ================================
# Imports
# ================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.impute import SimpleImputer
from minisom import MiniSom


# ================================
# Load Dataset
# ================================
data = pd.read_excel("/content/customer_data.xlsx")
print("Available columns:", data.columns.tolist())


# ================================
# Handle Missing Values
# ================================
# Numeric: median
num_imputer = SimpleImputer(strategy='median')
```

```
data[['age']] = num_imputer.fit_transform(data[['age']])

# Categorical: most frequent
cat_imputer = SimpleImputer(strategy='most_frequent')
data[['gender', 'payment_method']] = cat_imputer.fit_transform(
    data[['gender', 'payment_method']]
)


# ================================
# Encode Categorical Variables
# ================================
le_gender = LabelEncoder()
le_payment = LabelEncoder()

data['gender_enc'] = le_gender.fit_transform(data['gender'])
data['payment_enc'] = le_payment.fit_transform(data['payment_method'])


# ================================
# Feature Selection
# ================================
X_2d = data[['age', 'gender_enc']].values
X_som = data[['age', 'gender_enc', 'payment_enc']].values


# ================================
# Scaling
# ================================
scaler_2d = StandardScaler()
X_2d_scaled = scaler_2d.fit_transform(X_2d)

scaler_som = StandardScaler()
X_som_scaled = scaler_som.fit_transform(X_som)


# ================================
# K-Means Clustering
# ================================
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_2d_scaled)


# ================================
# Train SOM
# ================================
som = MiniSom(
    x=8, y=8,
    input_len=3,
    sigma=1.0,
    learning_rate=0.5,
    random_seed=42
)

som.random_weights_init(X_som_scaled)
som.train_random(X_som_scaled, num_iteration=1000)


# ================================
# Cluster SOM neurons using K-Means
# ================================
som_weights = som.get_weights().reshape(-1, 3)

som_kmeans = KMeans(n_clusters=3, random_state=42)
som_neuron_labels = som_kmeans.fit_predict(som_weights)


# ================================
# Assign SOM cluster to samples
# ================================
som_sample_labels = np.array([
    som_neuron_labels[som.winner(x)[0] * 8 + som.winner(x)[1]]
    for x in X_som_scaled
])


# ================================
# Evaluation Metrics
# ================================
silhouette_kmeans = silhouette_score(X_2d_scaled, kmeans_labels)
silhouette_som = silhouette_score(X_2d_scaled, som_sample_labels)
```

```
# ==============================
# Plotting (3 graphs)
# ==============================
fig, axes = plt.subplots(1, 3, figsize=(18, 5))


# ---- Plot 1: K-Means ----
axes[0].scatter(
    X_2d_scaled[:, 0],
    X_2d_scaled[:, 1],
    c=kmeans_labels,
    cmap='viridis',
    s=40
)
axes[0].set_title("K-Means Clustering (Age vs Gender)")
axes[0].set_xlabel("Age (scaled)")
axes[0].set_ylabel("Gender (encoded & scaled)")


# ---- Plot 2: SOM ----
axes[1].scatter(
    X_2d_scaled[:, 0],
    X_2d_scaled[:, 1],
    c=som_sample_labels,
    cmap='viridis',
    s=40
)
axes[1].set_title("SOM Clustering (Age vs Gender)")
axes[1].set_xlabel("Age (scaled)")
axes[1].set_ylabel("Gender (encoded & scaled)")


# ---- Plot 3: Metric Comparison ----
axes[2].bar(
    ['K-Means', 'SOM'],
    [silhouette_kmeans, silhouette_som]
)
axes[2].set_title("Silhouette Score Comparison")
axes[2].set_ylabel("Silhouette Score")


plt.tight_layout()
plt.show()


# ==============================
# Print Scores
# ==============================
print("Silhouette Scores:")
print(f"K-Means : {silhouette_kmeans:.3f}")
print(f"SOM     : {silhouette_som:.3f}")
```

```
Available columns: ['customer_id', 'gender', 'age', 'payment_method']
```




```
Silhouette Scores:
K-Means : 0.560
SOM     : 0.214
```